AMRITA VISHWA VIDYAPEETHAM

Course Code: 23ECE333

Course Name: Functional Verification

Component: Simulation-based Assignment

Date of Evaluation:

Academic Year: 2025-2026 (Even Semester)

Batch Number: 2

Name of the Students With Registration no.:

1) B. Lakshmi Dhatri - BL.EN.U4ECE23159
2) K. Charvi Reddy - BL.EN.U4ECE23161
3) M. Likhitha Reddy - BL.EN.U4ECE23168

Branch: ECE

Semester: VI Semester

Batch: 2023-2027

Faculty In-charge: Swaminadhan Rajula

Aim:
The aim of this project is to design and verify an **Index-Based Round Robin (IRR) Arbiter** using SystemVerilog for Network-on-Chip (NoC) router applications. The project focuses on providing fair and efficient arbitration among multiple requesters by rotating priority dynamically, thereby preventing starvation and ensuring equal access to shared resources. It also aims to achieve reduced hardware complexity, lower power consumption, and improved performance through an optimized arbitration architecture.

Base paper Details:

Title: Index-based Round-Robin Arbiter for NoC Routers

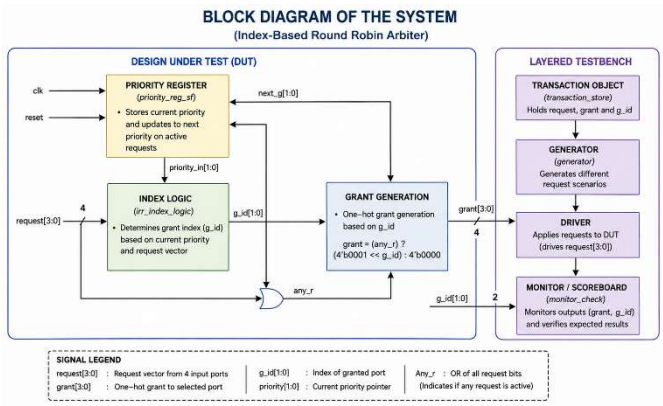
Authors: Masoud Oveis-Gharan and Gul N. Khan

Tools Used: Vivado

Theory:

1) **Introduction :**
   Arbiters are critical components in modern computing systems and buses (such as AXI or AHB). They manage multiple requests for a shared resource and grant access based on a specific algorithm. This project implements an Index-Based Round-Robin (IRR) Arbiter, which ensures that priority rotates among the requesting ports after each grant. This avoids starvation and provides fair access to resources.

2) **Block Diagram of the System :**



3) **Working principle :**
   - The arbiter continuously monitors the request signal.
   - irr_index_logic determines the index of the grant (g_id) based on the current priority vector and input requests.
   - The system computes next_g = g_id + 1'b1 to shift the priority to the next client in a round-robin manner.
   - Grant is asserted as a one-hot encoded vector corresponding to the calculated g_id.
   - The testbench uses tasks (driver, monitor_check) to apply multiple scenarios (all requests, no requests, partial requests) and verifies the grant against expected outcomes.

4) **Advantages:**
   - Fairness: Ensures all components are given equal opportunities to access shared resources.
   - Starvation Prevention: Prevents any single master from monopolizing the bus.
   - Hardware Efficiency: Simple one-hot calculation and priority updates.

5) **Disadvantages**
   Latency: May introduce a minor delay in calculating the next priority state depending on the critical path.

SystemVerilog Code for Design Block:

```
module priority_reg_sf(
   input clk,
   input reset,
   input any_r,
   input [1:0] next_g,
   output reg [1:0] priority_out
);
   always @(posedge clk or posedge reset) begin
      if (reset)
         priority_out <= 2'b00;
      else if (any_r)
         priority_out <= next_g;
      else
         priority_out <= priority_out;
   end
endmodule
module irr_index_logic(
   input [3:0] request,
   input [1:0] priority_in,
   output reg [1:0] g_id
);
   always @(*) begin
      case (priority_in)
         2'd0: begin
            if      (request[0]) g_id = 2'd0;
            else if (request[1]) g_id = 2'd1;
            else if (request[2]) g_id = 2'd2;
```

```
            else                g_id = 2'd3;
        end
        2'd1: begin
            if      (request[1]) g_id = 2'd1;
            else if (request[2]) g_id = 2'd2;
            else if (request[3]) g_id = 2'd3;
            else                g_id = 2'd0;
        end
        2'd2: begin
            if      (request[2]) g_id = 2'd2;
            else if (request[3]) g_id = 2'd3;
            else if (request[0]) g_id = 2'd0;
            else                g_id = 2'd1;
        end
        2'd3: begin
            if      (request[3]) g_id = 2'd3;
            else if (request[0]) g_id = 2'd0;
            else if (request[1]) g_id = 2'd1;
            else                g_id = 2'd2;
        end
        default: g_id = 2'd0;
    endcase
  end
endmodule
module irr_arbiter_top(
  input clk,
  input reset,
  input [3:0] request,
  output [3:0] grant,
  output [1:0] g_id
);
  wire [1:0] current_priority;
  wire [1:0] next_g;
  wire any_r;
  assign any_r = |request;
```

```
    priority_reg_sf PR_INST (
        .clk(clk),
        .reset(reset),
        .any_r(any_r),
        .next_g(next_g),
        .priority_out(current_priority)
    );
    irr_index_logic INDEX_LOGIC (
        .request(request),
        .priority_in(current_priority),
        .g_id(g_id)
    );
    assign grant = (any_r) ? (4'b0001 << g_id) : 4'b0000;
    assign next_g = g_id + 1'b1;
endmodule
module transaction_store;
    reg [3:0] req;
    reg [3:0] gnt;
    reg [1:0] gid;
endmodule
module generator(
    output reg [3:0] gen_request,
    output reg       gen_valid,
    input wire       clk
);
    initial begin
        gen_valid   = 0;
        gen_request = 4'b0000;
    end
endmodule
```

SystemVerilog Code for Layered Test Bench:

```
module tb_irr_layered;
    reg        clk;
    reg        reset;
    reg  [3:0] request;
```

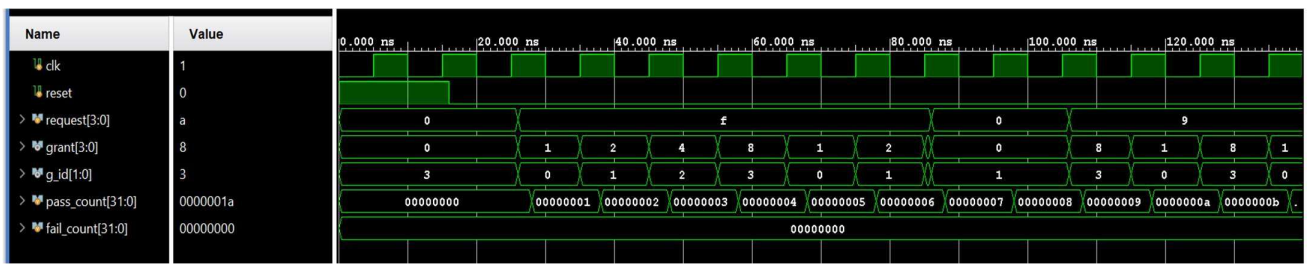
```
wire [3:0] grant;
wire [1:0] g_id;
integer pass_count;
integer fail_count;
irr_arbiter_top DUT (
    .clk    (clk),
    .reset  (reset),
    .request(request),
    .grant  (grant),
    .g_id   (g_id)
);
initial clk = 0;
always #5 clk = ~clk;
task driver;
    input [3:0] req;
    begin
        @(posedge clk);
        #1;
        request = req;
    end
endtask
task monitor_check;
    input [3:0] req;
    reg grant_ok;
    reg gid_ok;
    reg req_ok;
    begin
        #2; // wait for outputs to settle after clock edge
        if (req == 4'b0000) begin
            if (grant == 4'b0000) begin
                $display("[PASS] Time=%0t | Req=%b | Grant=%b | g_id=%0d | idle -> zero grant",
                        $time, req, grant, g_id);
                pass_count = pass_count + 1;
            end else begin
                $display("[FAIL] Time=%0t | Req=%b | Grant=%b | g_id=%0d | expected 0000",
```

```
                $time, req, grant, g_id);
            fail_count = fail_count + 1;
        end
    end else begin
        grant_ok = (grant != 0) && ((grant & (grant - 1)) == 0);
        gid_ok   = (grant == (4'b0001 << g_id));
        req_ok   = req[g_id];
        if (grant_ok && gid_ok && req_ok) begin
            $display("[PASS] Time=%0t | Req=%b | Grant=%b | g_id=%0d",
                $time, req, grant, g_id);
            pass_count = pass_count + 1;
        end else begin
            $display("[FAIL] Time=%0t | Req=%b | Grant=%b | g_id=%0d | one_hot=%b gid_ok=%b
req_ok=%b",
                $time, req, grant, g_id, grant_ok, gid_ok, req_ok);
            fail_count = fail_count + 1;
        end
    end
  end
  endtask
  task run_env;
    integer i;
    begin
      $display("------------------------------------------");
      $display(" Scenario 1: Req=1111 | 6 cycles");
      $display("  Expected   : 0001->0010->0100->1000");
      $display("------------------------------------------");
      for (i = 0; i < 6; i = i + 1) begin
        driver(4'b1111);
        monitor_check(4'b1111);
      end
      $display("------------------------------------------");
      $display(" Scenario 2: Req=0000 | 2 cycles");
      $display(" Expected  : Grant=0000, priority held");
      $display("------------------------------------------");
      for (i = 0; i < 2; i = i + 1) begin
```

```
            driver(4'b0000);
            monitor_check(4'b0000);
        end
        $display("-----------------------------------------");
        $display(" Scenario 3: Req=1001 | 6 cycles");
        $display(" Expected  : resumes from stored priority");
        $display("-----------------------------------------");
        for (i = 0; i < 6; i = i + 1) begin
            driver(4'b1001);
            monitor_check(4'b1001);
        end
        $display("-----------------------------------------");
        $display(" Scenario 4: Single requests");
        $display("-----------------------------------------");
        driver(4'b0001);    monitor_check(4'b0001);
        driver(4'b0010);    monitor_check(4'b0010);
        driver(4'b0100);    monitor_check(4'b0100);
        driver(4'b1000);  monitor_check(4'b1000);
        $display("-----------------------------------------");
        $display(" Scenario 5: Alternating 0101/1010");
        $display("-----------------------------------------");
        for (i = 0; i < 4; i = i + 1) begin
            driver(4'b0101); monitor_check(4'b0101);
            driver(4'b1010);  monitor_check(4'b1010);
        end
    end
endtask
initial begin
    pass_count = 0;
    fail_count = 0;
    reset   = 1;
    request = 4'b0000;
    @(posedge clk); #1;
    @(posedge clk); #1;
    reset = 0;
```

```
        run_env;
        $display("==========================================");
        $display("  SCOREBOARD FINAL REPORT");
        $display("  PASS  : %0d", pass_count);
        $display("  FAIL  : %0d", fail_count);
        $display("  TOTAL : %0d", pass_count + fail_count);
        if (fail_count == 0)
            $display("  STATUS: ALL TESTS PASSED");
        else
            $display("  STATUS: SOME TESTS FAILED");
        $display("==========================================");
        $finish;
    end
endmodule
```

Simulation Results :




```
--------------------------------------
 Scenario 1: Req=1111 | 6 cycles
 Expected  : 0001->0010->0100->1000
--------------------------------------
[PASS] Time=28000 | Req=1111 | Grant=0001 | g_id=0
[PASS] Time=38000 | Req=1111 | Grant=0010 | g_id=1
[PASS] Time=48000 | Req=1111 | Grant=0100 | g_id=2
[PASS] Time=58000 | Req=1111 | Grant=1000 | g_id=3
[PASS] Time=68000 | Req=1111 | Grant=0001 | g_id=0
[PASS] Time=78000 | Req=1111 | Grant=0010 | g_id=1
--------------------------------------
 Scenario 2: Req=0000 | 2 cycles
 Expected  : Grant=0000, priority held
--------------------------------------
[PASS] Time=88000 | Req=0000 | Grant=0000 | g_id=1 | idle -> zero grant
[PASS] Time=98000 | Req=0000 | Grant=0000 | g_id=1 | idle -> zero grant
--------------------------------------
 Scenario 3: Req=1001 | 6 cycles
 Expected  : resumes from stored priority
--------------------------------------
[PASS] Time=108000 | Req=1001 | Grant=1000 | g_id=3
[PASS] Time=118000 | Req=1001 | Grant=0001 | g_id=0
[PASS] Time=128000 | Req=1001 | Grant=1000 | g_id=3
[PASS] Time=138000 | Req=1001 | Grant=0001 | g_id=0
[PASS] Time=148000 | Req=1001 | Grant=1000 | g_id=3
[PASS] Time=158000 | Req=1001 | Grant=0001 | g_id=0
--------------------------------------
```

```
==========================================
  SCOREBOARD FINAL REPORT
  PASS  : 26
  FAIL  : 0
  TOTAL : 26
  STATUS: ALL TESTS PASSED
==========================================
$finish called at time : 278 ns : File "C:/Users/likhi/dhatri/dhatri.srcs/sources_1/new/dhatri1.v" Line 297
INFO: [USF-XSim-96] XSim completed. Design snapshot 'tb_irr_layered_behav' loaded.
INFO: [USF-XSim-97] XSim simulation ran for 1000ns
launch_simulation: Time (s): cpu = 00:00:06 ; elapsed = 00:00:09 . Memory (MB): peak = 1671.707 ; gain = 11.559
```

Conclusion:

The Index-Based Round Robin (IRR) Arbiter was successfully designed and verified using a layered SystemVerilog testbench. The arbiter correctly performed fair round-robin arbitration by dynamically updating priority and generating valid one-hot grant signals. Functional verification across multiple request scenarios confirmed correct operation, starvation prevention, and proper priority rotation. All test cases passed successfully with zero failures, demonstrating the reliability and efficiency of the proposed design.

References (if any other than base paper)

1. IEEE Standard for SystemVerilog—Unified Hardware Design, Specification, and Verification Language, IEEE Std 1800-2017.

2. Design of High-Performance Round-Robin Arbiters, Embedded Systems Letters, IEEE.